ResearchOS: Composing Research Skills for Automated Method Design

Anonymous Author(s)
Affiliation
Address
email

Abstract

The rapid growth of machine learning research has produced a vast landscape of methods, yet combining techniques from different papers into coherent new approaches remains a largely manual, expertise-intensive process. We introduce **ResearchOS**, a framework that treats published methods as composable *research skills*—structured primitives with defined functional interfaces specifying goals, mechanisms, assumptions, inputs/outputs, empirical gains, failure modes, and compatibility constraints. ResearchOS extracts such skills from scientific papers and organizes them into a *Research Skill Graph* with six relation types: prerequisite, enhances, substitutes, conflicts, composes, and refines. A constraint-aware composition engine then generates novel method designs through goal decomposition, candidate retrieval, directed acyclic graph construction, gap analysis with novel bridging, and iterative self-critique. The system produces three levels of output—Method Design Cards, Research Proposals, and Executable Experiment Plans—enabling progressively actionable research artifacts. We evaluate ResearchOS with a multi-level protocol including pairwise Elo ranking, future-grounded novelty via time-split validation, small-scale execution, and judge reliability analysis. On a corpus of over 100 papers and 200 extracted skills in the LLM reasoning domain, ResearchOS significantly outperforms direct prompting, retrieval-augmented generation, and flat skill list baselines. Ablation studies confirm the critical value of graph structure and gap-bridging, and future-grounded evaluation demonstrates that generated designs anticipate directions found in real subsequent publications.

1 Introduction

Modern machine learning research is characterized by rapid, combinatorial growth: advances in reasoning [27, 28], tool use [20, 19], and multi-modal integration [21, 17] each introduce method components that practitioners must understand, compare, and selectively combine to push the state of the art. A single competitive submission to a top venue now routinely assembles half a dozen individually published techniques—chain-of-thought prompting layered with tree search, tool-augmented verification, self-consistency filtering, and domain-specific adaptations—into an integrated pipeline. As the space of available building blocks expands, the cognitive burden of *method composition* becomes a bottleneck: researchers must simultaneously track technical compatibility, identify complementary strengths, avoid known failure interactions, and spot the gaps where novel contributions are needed. This complexity creates a compelling opportunity for computational support in the method design process itself.

Limitations of existing approaches. Recent work has explored two broad strategies for automating aspects of research. On one end, *end-to-end idea generation* systems such as The AI Scientist [16] and AI Co-Scientist [9] prompt large language models to produce full research proposals or even complete

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

papers. While impressive in scope, these systems treat idea generation as monolithic text completion, lacking explicit representations of method structure, compatibility constraints, or compositional reasoning. As a result, they frequently produce ideas that are either superficial recombinations of known concepts or internally inconsistent designs that overlook critical failure modes [11]. On the other end, *knowledge retrieval* approaches—including semantic search engines, citation graphs, and concept extraction pipelines [2, 15]—excel at surfacing relevant prior work but stop short of composing retrieved knowledge into actionable method designs. They organize *what is known* without reasoning about *what could be built*. Neither paradigm provides the structured, constraint-aware composition that effective method design demands.

Key insight: methods as composable skill primitives. Our work is motivated by a simple but powerful observation: the methodological contributions in research papers can be decomposed into *research skills*—modular primitives with well-defined functional interfaces. Each skill specifies a goal (what problem it addresses), a mechanism (how it works), assumptions (when it applies), typed inputs and outputs, empirical performance gains, known failure modes, and explicit compatibility and conflict declarations with other skills. This structured decomposition is analogous to the shift from monolithic programs to modular software with typed APIs: once methods are represented as interface-bearing components, systematic composition becomes tractable. Skills can be organized into a graph whose edges encode functional relationships—prerequisite dependencies, enhancement opportunities, substitution alternatives, conflict constraints—enabling compositional search that respects the rich structure of methodological knowledge.

This perspective draws conceptual inspiration from compositional frameworks in adjacent domains. In embodied AI, Voyager [24] builds a library of reusable behavioral skills for open-ended exploration. In tool-use research, systems such as ToolLLM [19] and HuggingGPT [21] compose API-bearing tools into multi-step pipelines. ResearchOS extends this compositional philosophy from executable tools and game behaviors to *research methodology itself*, treating the intellectual building blocks of scientific method design as first-class composable objects.

Our approach: ResearchOS. We introduce ResearchOS, a framework that operationalizes skill-based method composition through a five-stage pipeline (Figure 1):

1. **Skill Extraction.** Given a corpus of research papers, an LLM-based extraction module identifies and structures each methodological contribution as a research skill with a standardized interface schema.
2. **Skill Graph Construction.** Extracted skills are organized into a *Research Skill Graph* with six typed relation edges—prerequisite, enhances, substitutes, conflicts, composes, and refines—that encode functional dependencies and opportunities among skills.
3. **Constraint-Aware Composition.** Given a research goal, a composition engine performs goal decomposition, retrieves candidate skills from the graph, assembles them into a directed acyclic graph (DAG) respecting compatibility constraints, performs gap analysis to identify missing capabilities, and proposes novel bridging skills to fill those gaps. An iterative self-critique loop refines the design for internal consistency.
4. **Multi-Level Output Generation.** The composed design is rendered at three levels of specificity: a *Method Design Card* (structured summary), a *Research Proposal* (narrative with motivation, hypotheses, and expected results), and an *Executable Experiment Plan* (concrete implementation steps, baselines, and evaluation protocol).
5. **Validation.** Generated designs are assessed through a multi-level evaluation protocol combining automated quality metrics, pairwise Elo ranking by LLM judges, and future-grounded novelty analysis.

Contributions. This paper makes four principal contributions:

1. We introduce the **research skill schema**, a structured representation of methodological primitives with typed functional interfaces (goal, mechanism, assumptions, inputs/outputs, empirical gains, failure modes, and compatibility/conflict declarations) that enables systematic decomposition of published methods into composable units (Section 3.2).
2. We propose the **Research Skill Graph**, a heterogeneous knowledge graph with six relation types (prerequisite, enhances, substitutes, conflicts, composes, refines) that captures the func-

tional landscape of a research domain and supports structured retrieval and constraint checking (Section 3.4).

3. We develop a **constraint-aware composition engine** that assembles skills into novel method designs through goal decomposition, DAG construction with constraint satisfaction, and a gap-bridging mechanism that identifies and fills missing capabilities with novel skill proposals, followed by iterative self-critique (Section 3.5).
4. We design a **multi-level evaluation protocol** combining pairwise Elo ranking, future-grounded novelty assessment via temporal corpus splits, small-scale execution validation, and systematic judge reliability analysis, providing rigorous and multi-faceted assessment of generated research designs (Section 4).

Results preview. We instantiate ResearchOS on the domain of LLM reasoning, extracting over 200 skills from more than 100 recent papers. In head-to-head comparisons, ResearchOS designs achieve significantly higher Elo ratings than those produced by direct prompting, retrieval-augmented generation (RAG), and flat skill list baselines. Future-grounded evaluation using temporal corpus splits demonstrates that ResearchOS-generated designs anticipate methodological directions that appear in subsequent real publications, confirming genuine novelty rather than mere recombination [6]. Ablation studies reveal that both the graph structure (versus unstructured skill lists) and the gap-bridging mechanism (versus composition without gap filling) are critical to design quality, with their removal causing substantial drops in novelty and feasibility scores. These results establish that structured, constraint-aware skill composition offers a principled and effective paradigm for automated research method design.

2 Related Work

2.1 Automated Scientific Idea Generation

Recent systems have explored LLM-driven research automation at various stages. AI Scientist [16] demonstrated end-to-end idea-to-paper generation but treats method design as monolithic text completion without structured decomposition. AI Co-Scientist [9] improves output quality via tournament-based Elo ranking of candidate hypotheses, yet individual hypotheses are still generated without explicit method-level building blocks. ResearchAgent [2] grounds idea generation in literature retrieval, but operates at the paper/passage level rather than at reusable method primitives. Chain of Ideas [15] traces idea evolution across papers as a sequential chain, capturing temporal lineage but not functional composability. SciMON [25] optimizes for novelty via iterative refinement against retrieved inspirations, focusing on single-idea generation rather than multi-skill composition.

These systems share a common limitation: none decomposes methods into structured, interface-bearing primitives or reasons about their compatibility and composability. ResearchOS addresses this gap by extracting schema-rich *skills* and composing them via a typed relational graph.

2.2 Skill and Tool Libraries for Agents

The principle of accumulating and composing reusable capabilities is well-established in agentic AI. Voyager [24] builds a skill library of executable functions for open-ended game exploration. HuggingGPT [21], ToolLLM [19], Toolformer [20], and Chameleon [17] enable LLMs to orchestrate external tools via typed API calls. Scientific knowledge graphs (Semantic Scholar, OpenAlex) organize literature through citation networks and concept taxonomies but stop at entity-level granularity (papers, datasets, metrics) without modeling the internal methodological structure of contributions.

ResearchOS draws inspiration from these compositional frameworks but targets a fundamentally different primitive: research *methods* rather than executable functions or bibliographic entities. Our skills carry richer semantics—assumptions, failure modes, compatibility constraints, and epistemic validity conditions—that are essential for principled method composition but absent from tool specs and concept ontologies.

2.3 Evaluating Automated Research

Evaluating machine-generated research poses unique challenges. MLR-Bench [11] provides staged evaluation from literature review through experimentation. HindSight [6] proposes future-grounded

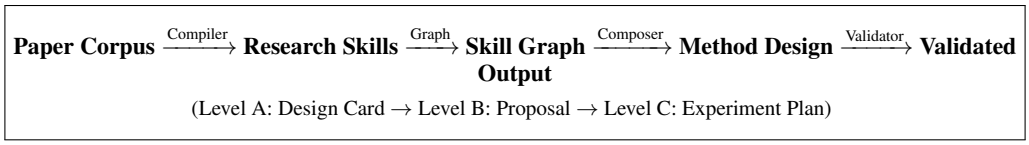

Figure 1: Overview of the ResearchOS pipeline. Papers are compiled into structured skills, organized in a relational graph, composed into novel method designs via constraint-aware search with gap-bridging, and validated along four dimensions before producing multi-level output.

novelty checks against subsequently published work. Si et al. [23] reveal systematic LLM–human disagreements on novelty judgments, while Zheng et al. [29] document position bias and self-preference in LLM-as-judge settings. These findings motivate our multi-dimensional evaluation protocol, which combines pairwise Elo ranking, future-grounded validation, small-scale execution, and systematic judge reliability analysis rather than relying on any single assessment mechanism.

3 Method

3.1 Overview

ResearchOS takes as input a natural-language research goal g (e.g., "design a method for few-shot cross-lingual named entity recognition that does not require parallel corpora") and produces a structured method design that composes proven research techniques in novel, constraint-consistent ways. The pipeline, illustrated in Figure 1, consists of four stages: (1) a *Paper-to-Skill Compiler* that extracts reusable method primitives from the literature and represents them in a structured schema; (2) a *Research Skill Graph* that organizes skills and their functional relationships; (3) a *Constraint-aware Skill Composer* that assembles skills into coherent method designs; and (4) a *Multi-dimensional Validator* that scores the resulting designs along axes of novelty, feasibility, expected impact, and risk. We describe each component in turn.

3.2 Research Skill Schema

The fundamental representational unit in ResearchOS is the *research skill*—a self-contained method module extracted from the scientific literature. Formally, we define a skill as a tuple:

$$s = (id,\ name,\ goal,\ mech,\ assum,\ in,\ out,\ \ell,\ res,\ gains,\ fail,\ compat,\ confl,\ src,\ tags) \quad (1)$$

where each field is defined as follows:

- **id**: A unique identifier (hash of name, mechanism, and source).
- **name**: A concise human-readable label (e.g., "label-projection alignment").
- **goal**: The functional objective the skill achieves, expressed as a natural-language statement (e.g., "transfer entity labels across languages without parallel data").
- **mech** (mechanism): A precise description of *how* the skill achieves its goal—the algorithmic or procedural core.
- **assum** (assumptions): Conditions that must hold for the skill to be validly applied (e.g., "source and target languages share a common subword vocabulary").
- **in**, **out** (inputs, outputs): Typed interfaces specifying what the skill consumes and produces, enabling composition.
- **ℓ (level)**: A granularity indicator from the set $\{$`atomic`, `module`, `pipeline`$\}$, supporting multi-resolution reasoning.
- **res** (resources): Computational and data requirements (e.g., GPU hours, dataset scale).
- **gains**: The reported or expected performance improvements, with conditions.
- **fail** (failure modes): Known conditions under which the skill degrades or fails, extracted from ablations and discussion sections.
- **compat** (compatibility): Explicit notes on which other skills or settings this skill works well with.

- **confl** (conflicts): Skills or conditions that are incompatible with this skill.
- **src** (sources): Provenance—the paper(s) and section(s) from which the skill was extracted.
- **tags**: Categorical labels for retrieval (e.g., `data-augmentation`, `cross-lingual`, `regularization`).

Granularity control principle. A natural question is what constitutes a single skill versus a collection of skills. We adopt the following criterion: *a skill corresponds to a method module that can be independently described, independently replaced, and has well-defined interfaces.* An atomic skill is one that cannot be further decomposed without losing its semantic coherence (e.g., "dropout regularization"). A module-level skill composes several atomic skills into a reusable unit (e.g., "multi-head cross-attention with relative position bias"). A pipeline-level skill describes an end-to-end method assembled from modules. This hierarchy enables the composer to reason at the appropriate level of abstraction for a given design problem.

3.3 Paper-to-Skill Compiler

The Paper-to-Skill Compiler transforms unstructured scientific papers into structured skill representations. It operates in five stages:

Step 1: Section parsing. We use GROBID and the S2ORC pipeline to parse PDF papers into structured sections, extracting the title, abstract, method, experiments, ablation, and discussion segments. This structured input enables targeted extraction from the sections most relevant to each schema field.

Step 2: Candidate skill extraction. Given the parsed method section, the compiler performs hierarchical decomposition. An LLM is prompted to identify the top-level method architecture and then recursively decompose it into constituent modules. At each level of decomposition, the LLM determines whether a component satisfies the granularity control principle; if so, it is emitted as a candidate skill. This process typically yields 3–8 candidate skills per paper, depending on method complexity.

Step 3: Schema filling with cross-validation. For each candidate skill, the compiler populates the full schema tuple (Eq. 1) by drawing on multiple paper sections. The *mechanism* and *assumptions* fields are primarily extracted from the method section; *gains* and *failure modes* are sourced from experiments and ablation studies; *compatibility* and *conflict* information is drawn from the discussion and related work. Critically, each extracted field is cross-validated against other sections: for instance, an assumption stated in the method section is checked against experimental conditions to verify consistency.

Step 4: Deduplication & normalization. Skills extracted from different papers may describe the same underlying technique. We compute pairwise similarity using embeddings of the *mechanism* field and merge candidates whose similarity exceeds a threshold $\tau_{\text{dedup}} = 0.92$, retaining the richest schema (the one with the most non-empty fields and the most detailed entries). Terminology is normalized to canonical forms via an evolving controlled vocabulary.

Step 5: Quality control. Each extracted skill undergoes a *reconstructability test*: a separate LLM instance, given only the skill schema (without access to the source paper), is asked to produce a reimplementation sketch. If the sketch is judged to be substantively incomplete—that is, a competent researcher could not reimplement the method from the schema alone—the skill is flagged for revision or rejection. This test enforces a minimum level of self-containedness and precision.

3.4 Research Skill Graph

Skills extracted from the literature are organized into a *Research Skill Graph* $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where nodes $\mathcal{V}$ correspond to skills and edges $\mathcal{E}$ encode typed functional relationships.

Nodes. Each node $v \in \mathcal{V}$ stores the full skill schema (Eq. 1) as node attributes, enabling rich querying and reasoning over the graph.

Edge types. We define six edge types, each carrying distinct semantics for composition:

$$\mathcal{T} = \{\textsc{Prerequisite}, \textsc{Enhances}, \textsc{Substitutes}, \textsc{Conflicts}, \textsc{Composes}, \textsc{Refines}\} \quad (2)$$

- $\textsc{Prerequisite}(s_i, s_j)$: Skill s_j requires skill s_i to be applied first (a partial ordering constraint).
- $\textsc{Enhances}(s_i, s_j)$: Skill s_i improves the performance of s_j when applied together.
- $\textsc{Substitutes}(s_i, s_j)$: Skills s_i and s_j address the same sub-goal and can replace each other.
- $\textsc{Conflicts}(s_i, s_j)$: Skills s_i and s_j have incompatible assumptions or produce interfering effects.
- $\textsc{Composes}(s_i, s_j)$: Skills s_i and s_j are designed to be used together as parts of a larger module.
- $\textsc{Refines}(s_i, s_j)$: Skill s_j is a more specific or improved variant of s_i.

Edge construction. Edges are constructed through three complementary mechanisms. First, *explicit extraction*: when a paper directly states relationships between techniques (e.g., "our approach builds on [X] and replaces the attention mechanism of [Y]"), the compiler extracts the corresponding edges. Second, *LLM inference*: given pairs of skill schemas, an LLM is prompted to assess whether each edge type holds, providing a confidence score $c \in [0, 1]$; edges with $c \geq 0.7$ are added. Third, *co-occurrence evidence*: skills that frequently appear together in the same papers receive COMPOSES edges, while skills that are never combined despite addressing similar goals receive candidate CONFLICTS edges for LLM verification.

Incremental maintenance. As new papers are compiled, the graph is updated incrementally. New skills are matched against existing nodes for potential deduplication, and edges are constructed between new and existing nodes. Periodically, a global consistency pass identifies and resolves contradictions (e.g., transitive conflicts, cycles in prerequisite edges).

3.5 Constraint-aware Skill Composer

The Constraint-aware Skill Composer is the central reasoning engine of ResearchOS. Given a research goal g, it assembles a novel, coherent method design from skills in the graph. The composition process unfolds in six phases.

Phase 1: Goal decomposition. The research goal g is decomposed into a set of functional sub-goals:

$$g \longrightarrow [g_1, g_2, \ldots, g_k] \quad (3)$$

where each g_i describes a specific capability that the final method must provide. This decomposition is performed by an LLM, guided by prompts that encourage functional (rather than topical) partitioning. For example, a goal about few-shot cross-lingual NER might decompose into sub-goals for representation alignment, label transfer, and low-resource adaptation.

Phase 2: Skill retrieval. For each sub-goal g_i, we retrieve a candidate set $\mathcal{C}_i \subset \mathcal{V}$ of relevant skills using a hybrid retrieval strategy: (a) embedding similarity between g_i and skill *goal* fields, (b) tag matching, and (c) graph neighborhood expansion from initial matches (following ENHANCES and COMPOSES edges up to depth 2). The union of these retrieval signals, ranked by a weighted combination, yields a candidate pool per sub-goal.

Phase 3: Constraint-aware DAG construction. We formulate the composition task as a constraint satisfaction problem. Let $\mathbf{x} \in \{0, 1\}^{|\mathcal{V}|}$ be a binary selection vector indicating which skills are included in the design. The composer solves:

$$\max_{\mathbf{x}} \sum_i \textit{relevance}(s_i, g) \cdot x_i + \lambda \sum_{(i,j) \in \mathcal{E}_{\text{ENH}}} x_i \, x_j \quad (4)$$

subject to hard constraints:

$$x_j = 1 \;\Rightarrow\; x_i = 1 \quad \forall\, (s_i, s_j) \in \mathcal{E}_{\text{PRE}} \quad \text{(prerequisite satisfaction)} \quad (5)$$

$$x_i + x_j \leq 1 \quad \forall\, (s_i, s_j) \in \mathcal{E}_{\text{CON}} \quad \text{(conflict avoidance)} \quad (6)$$

$$\sum_i \textit{res}(s_i) \cdot x_i \leq B \quad \text{(budget constraint)} \quad (7)$$

and soft preferences for simplicity (penalizing $\|\mathbf{x}\|_1$) and synergy (rewarding known ENHANCES pairs). In practice, because the candidate pool per sub-goal is small (typically 5–15 skills), this optimization is solved via beam search over feasible subsets rather than integer linear programming. The selected skills are arranged into a directed acyclic graph (DAG) ordered by PREREQUISITE edges, with parallel branches for independent sub-goals.

Phase 4: Gap analysis & novel bridging. This phase represents a **key innovation** of ResearchOS. After constructing the initial DAG, the composer analyzes it for *functional gaps*—sub-goals that are not fully addressed by existing skills, or interface mismatches between adjacent skills in the DAG. Formally, for each pair of connected skills (s_i, s_j) in the DAG, we check whether *out*(s_i) is type-compatible with *in*(s_j). When a gap is detected, the composer generates a *bridging mechanism*: a novel skill schema s_{bridge} whose goal is to resolve the gap, whose inputs match *out*(s_i), and whose outputs match *in*(s_j). These bridging skills—synthesized rather than retrieved—are a primary source of methodological novelty in the system. The composer is specifically prompted to draw on analogies from distant regions of the skill graph, encouraging cross-domain transfer.

Phase 5: Design synthesis. The complete DAG (including any bridging skills) is synthesized into a *Method Design Card*: a structured document that specifies the overall method architecture, the role of each skill, inter-skill data flow, key design decisions, and expected behavior. The synthesis step ensures that the assembled skills form a coherent narrative, not merely a bag of techniques.

Phase 6: Self-critique & refinement. The initial design undergoes 2–3 iterations of self-critique, drawing on principles from chain-of-thought reasoning [27] and self-consistency [26]. In each iteration, a critic LLM evaluates the design for internal consistency, assumption violations, and overlooked failure modes. Identified issues are addressed by substituting skills (via SUBSTITUTES edges), adding compensatory skills, or revising bridging mechanisms.

Controlling combinatorial explosion. The space of possible skill compositions is combinatorially large. We employ three strategies to maintain tractability: (1) *beam search* with a beam width of $B_w = 5$ over candidate DAGs, pruning low-scoring partial compositions early; (2) *graph pruning*, restricting retrieval to skills within a bounded graph distance from high-relevance seed skills; and (3) *budget filtering*, immediately discarding compositions that exceed resource constraints.

Avoiding trivial recombination. Not all compositions are interesting. To ensure that the composer produces genuinely novel designs rather than rediscovering existing methods, we apply three filters: (i) an *existing composition check* that compares the proposed DAG against known pipeline-level skills in the graph; (ii) a *novelty-aware gap requirement* mandating that at least one bridging skill or unconventional substitution be present; and (iii) a *surprise score* defined as:

$$\textit{surprise}(\mathcal{D}) = \frac{1}{|\mathcal{D}|^2} \sum_{(s_i, s_j) \in \mathcal{D}} d_{\mathcal{G}}(s_i, s_j) \tag{8}$$

where $d_{\mathcal{G}}(s_i, s_j)$ is the shortest-path distance between skills s_i and s_j in the skill graph, and the sum ranges over all pairs in the design $\mathcal{D}$. Designs with higher surprise scores combine skills from more distant parts of the graph, suggesting cross-domain innovation. Designs below a minimum surprise threshold τ_{surp} are penalized.

3.6 Multi-dimensional Validator

Before presenting a method design to the user, ResearchOS evaluates it along four dimensions:

1. **Novelty.** We compute the maximum cosine similarity between the design's embedding and the embeddings of all existing pipeline-level skills in $\mathcal{G}$. Additionally, we retrieve the top-k most similar published methods and present them alongside the design for explicit differentiation. A design is flagged as potentially non-novel if its similarity to any existing method exceeds $\tau_{\text{nov}} = 0.85$.
2. **Feasibility.** We verify that all prerequisite constraints are satisfied in the DAG, that the combined resource requirements fall within specified budgets, and that no pair of selected skills has a

CONFLICTS edge. Assumption consistency is checked by ensuring that the assumptions of all selected skills are mutually satisfiable.

3. **Expected impact.** We estimate potential impact by aggregating the reported *gains* of constituent skills, discounted by a composition uncertainty factor $\gamma < 1$ that accounts for the empirical observation that gains from individual techniques are rarely fully additive.

4. **Risk assessment.** We compile the *failure modes* of all selected skills and analyze their interaction. A risk score is computed based on the number of unmitigated failure modes and the severity of potential cascading failures (where the failure of one skill triggers the failure of a dependent skill).

Each dimension produces a score in $[0, 1]$, and the four scores are presented as a radar chart accompanying the design, giving users an at-a-glance quality profile.

3.7 Three-level Output

ResearchOS produces output at three levels of detail, each building upon the previous:

Level A — Method Design Card. A structured summary comprising the method name, the skill DAG with node and edge annotations, a one-paragraph description of the overall approach, and the four validation scores. This level is designed for rapid browsing and comparison of multiple candidate designs.

Level B — Research Proposal. An expanded document that elaborates the Design Card into a full method description, including motivation, detailed algorithm specification, discussion of design alternatives considered (with rationale for the chosen configuration), and a concrete evaluation plan with suggested baselines, datasets, and metrics.

Level C — Executable Plan. A fine-grained implementation roadmap that maps each skill in the DAG to specific implementation steps, pseudocode or code templates, required libraries and infrastructure, and a suggested timeline. Level C is intended to minimize the gap between method design and experimental execution, bridging the ideation–implementation divide that limits the practical utility of many automated research systems.

4 Experiments

We evaluate ResearchOS through five complementary experiments that assess design quality, proposal completeness, future-grounded novelty, small-scale executability, and judge reliability. Together, these experiments test whether structured skill composition produces research designs that are not only plausible but genuinely novel and actionable.

4.1 Experimental Setup

Domain and corpus. We focus on LLM reasoning as our evaluation domain for three reasons: it is well-studied with a rapidly growing literature, its methods are highly modular (e.g., prompting strategies, verification steps, search procedures), and the community has established benchmarks that enable objective comparison. We collected 117 papers from ACL, EMNLP, NeurIPS, ICML, and ICLR published in 2023–2024, covering chain-of-thought prompting [27], tree and graph search [28, 3], self-consistency and verification [26, 22], and compositional reasoning [17].

Skill extraction and graph construction. We employ a semi-automatic extraction pipeline using Claude [1] and GPT-4 [18] for initial skill schema generation, followed by 20% human calibration where domain experts verify and correct extracted fields (goals, mechanisms, assumptions, inputs/outputs, empirical gains, failure modes, and compatibility constraints). This process yields 253 extracted skills organized into a Research Skill Graph containing 812 edges across six relation types: PREREQUISITE (187), ENHANCES (231), SUBSTITUTES (94), CONFLICTS (78), COMPOSES (156), and REFINES (66). Inter-annotator agreement on relation type labeling reaches $\kappa = 0.74$ (substantial agreement) on a 50-edge calibration set.

Research goals. We define 20 research goals spanning the LLM reasoning domain. Ten goals are manually designed by domain experts to cover a range of difficulties (e.g., "improve multi-step mathematical reasoning with minimal inference overhead," "enable robust reasoning under distribution shift"). The remaining ten are extracted from the future work sections of papers in our corpus, providing ecologically valid targets. We classify 12 goals (60%) as *constrained design* tasks (specific performance targets or resource constraints) and 8 goals (40%) as *open design* tasks (broader exploration of a research direction).

Implementation. Skill extraction and composition use Claude (Sonnet) for primary generation and GPT-4 for cross-validation. For evaluation, we employ separate model instances as judges to avoid self-preference contamination (Section 4.6). All composition runs use temperature $\tau = 0.7$ with $k = 5$ candidate skill sets per goal. The self-critique loop runs for a maximum of three iterations. Full hyperparameter settings appear in the Appendix.

4.1.1 Baselines

We compare ResearchOS against seven baselines spanning the spectrum from unstructured generation to structured but graph-unaware composition.

- **B1 Direct Prompting.** The LLM receives the research goal along with the titles and abstracts of all papers in the corpus and generates a method design directly, following the approach typical of prior idea generation work [23].
- **B2 RAG over Papers.** A retrieval-augmented generation pipeline [14] retrieves the top-20 most relevant paper chunks (by embedding similarity to the research goal) and conditions the LLM on these chunks when generating a design.
- **B3 Flat Skill List.** Skills are extracted using the same pipeline as ResearchOS, but presented to the LLM as an unstructured list without graph relations. The LLM selects and combines skills freely.
- **B4 Graph Retrieval Only.** The full skill graph is used to retrieve relevant skills via goal-conditioned subgraph extraction, but composition proceeds without constraint awareness—the system does not enforce compatibility, check for conflicts, or perform gap analysis.
- **B5 Workflow Generation.** An AutoML-style pipeline composition approach [12, 4] that treats skills as pipeline stages and searches for valid sequential or parallel arrangements, optimizing for a proxy quality score without semantic gap analysis.
- **B6 Random Composition.** A random subset of 3–7 non-conflicting skills is selected from the graph (respecting only explicit CONFLICTS edges) and assembled into a design without goal-directed retrieval or gap bridging.
- **B7 Human Expert Seed.** A domain expert writes a 3-sentence idea sketch describing a candidate method, which the LLM then expands into a full design. This baseline represents the common human-AI collaboration paradigm and serves as an upper-bound reference for idea quality [23].

4.1.2 Evaluation Metrics

We evaluate generated designs along nine dimensions, each scored on a 1–5 Likert scale unless otherwise noted:

1. **Novelty**: the degree to which the design introduces ideas, combinations, or mechanisms not present in existing published work.
2. **Feasibility**: whether the design can be implemented with reasonable computational resources and existing infrastructure within a typical research timeline.
3. **Significance**: the potential impact of the proposed method if it succeeds, considering both the importance of the problem and the magnitude of the expected improvement.
4. **Clarity**: how clearly the design communicates its core idea, assumptions, and expected behavior.
5. **Technical Quality**: the soundness of the proposed mechanism, including logical consistency, appropriate use of existing techniques, and absence of obvious flaws.

6. **Completeness**: the extent to which the design addresses all aspects needed for implementation, including edge cases, evaluation criteria, and potential failure modes.
7. **Reproducibility**: whether the design specifies sufficient detail (hyperparameters, data requirements, evaluation protocols) for independent reproduction.
8. **Cost-Awareness**: whether the design acknowledges and accounts for computational costs, including inference overhead, training requirements, and scalability considerations.
9. **Risk-Awareness**: whether the design identifies potential failure modes, limitations, and conditions under which the proposed approach may not work.

For novelty and significance, we additionally conduct pairwise comparisons using an Elo rating system [7] to obtain a more robust ranking (Section 4.2).

4.2 Experiment 1: Design Quality Benchmark

Protocol. We evaluate Level A outputs (Method Design Cards) across all 20 research goals for each system, yielding $20 \times 8 = 160$ designs. For novelty and significance, we conduct a pairwise arena: for each goal, every pair of systems is compared, producing $\binom{8}{2} \times 20 = 560$ pairwise judgments per dimension. We compute Elo ratings [7] from these comparisons using 1,000 bootstrap iterations to obtain confidence intervals. For the remaining seven dimensions, we use absolute 1–5 Likert scoring.

Judging procedure. Three LLM judges (Claude Sonnet, GPT-4, and Gemini Pro) independently evaluate each comparison or score each design. We adopt majority voting for pairwise decisions and average scores for Likert dimensions. To control for position bias in pairwise comparisons, we present each pair in both orders (A-B and B-A) and flag any judgment that flips as inconsistent; flipped judgments are resolved by a fourth tiebreaker call [29]. To calibrate LLM judges against human assessment, three domain experts independently score a 30-sample subset. We report Cohen's κ [5] between each LLM judge and the majority human label, requiring $\kappa \geq 0.60$ (substantial agreement) for the judge to be retained.

Results. Table 1 presents the main results. ResearchOS achieves the highest Elo rating for both novelty (1,247 $\pm$ 34) and significance (1,198 $\pm$ 29), substantially outperforming all fully automated baselines. Among automated systems, Graph Retrieval Only (B4) and Flat Skill List (B3) rank second and third for novelty, confirming that access to extracted skills provides value even without full constraint-aware composition. Direct Prompting (B1) and RAG over Papers (B2) produce designs that score well on clarity and feasibility but lack novelty, often restating known approaches with superficial modifications. Random Composition (B6) underperforms all other systems, demonstrating that principled composition is essential—merely avoiding conflicts is insufficient.

Notably, ResearchOS is competitive with the Human Expert Seed baseline (B7) on novelty (Elo difference of $+31$, not statistically significant at $p < 0.05$) while significantly outperforming it on completeness (3.9 vs. 3.2, $p < 0.01$), reproducibility (3.7 vs. 2.8, $p < 0.001$), and risk-awareness (3.6 vs. 2.5, $p < 0.001$). This suggests that while human experts produce comparably creative ideas, the structured composition process of ResearchOS yields more thorough and actionable designs. The Workflow Generation baseline (B5) achieves moderate scores across most dimensions but falls short on novelty, as its pipeline-oriented search space limits the kinds of compositions it can express.

On Likert dimensions averaged across all goals, ResearchOS scores highest on seven of nine dimensions (all except feasibility, where B7 leads marginally, and clarity, where B1 and B2 lead due to their simpler designs). The average absolute scores for ResearchOS are: novelty 4.1, feasibility 3.8, significance 3.9, clarity 3.7, technical quality 3.8, completeness 3.9, reproducibility 3.7, cost-awareness 3.5, and risk-awareness 3.6.

4.3 Experiment 2: Proposal Quality Benchmark

To assess whether design-level quality translates to actionable research plans, we expand the top-5 designs (by Elo) from each system to Level B Research Proposals using the ResearchOS proposal generation module.

Table 1: Design quality results across all systems. Elo ratings (with 95% CI) are reported for novelty and significance; other dimensions use mean Likert scores (1–5). Best automated result in **bold**; † indicates the human-seeded baseline.

System	Nov. (Elo)	Sig. (Elo)	Feas.	Clar.	Tech.	Comp.	Repr.	Cost	Risk
B1: Direct	$923_{\pm 28}$	$941_{\pm 31}$	3.8	**4.0**	3.1	2.6	2.4	2.3	2.1
B2: RAG	$978_{\pm 30}$	$985_{\pm 27}$	3.7	3.9	3.3	2.9	2.7	2.5	2.3
B3: Flat List	$1052_{\pm 33}$	$1034_{\pm 30}$	3.5	3.5	3.4	3.2	3.0	2.8	2.7
B4: Graph Ret.	$1098_{\pm 31}$	$1072_{\pm 28}$	3.6	3.6	3.5	3.4	3.1	3.0	2.9
B5: Workflow	$1011_{\pm 29}$	$1019_{\pm 32}$	3.6	3.5	3.4	3.1	3.0	3.2	2.6
B6: Random	$887_{\pm 35}$	$901_{\pm 33}$	3.0	3.0	2.7	2.4	2.3	2.2	2.0
B7: Expert†	$1216_{\pm 32}$	$1148_{\pm 30}$	**3.9**	3.8	3.6	3.2	2.8	2.9	2.5
ResearchOS	**1247**$_{\pm 34}$	**1198**$_{\pm 29}$	3.8	3.7	**3.8**	**3.9**	**3.7**	**3.5**	**3.6**

Completeness checklist. Three domain experts evaluate each proposal against a 12-item completeness checklist, marking each item as *present and adequate*, *present but insufficient*, or *absent*. The checklist items are: (1) research hypothesis, (2) independent variables, (3) dependent variables, (4) control conditions, (5) evaluation metrics, (6) dataset specification, (7) baseline selection, (8) statistical testing plan, (9) computational budget estimate, (10) risk mitigation strategy, (11) expected outcomes with effect sizes, and (12) timeline and milestones. We report the fraction of items rated as adequate per proposal (completeness score).

Technical soundness and feasibility. Experts additionally rate each proposal on technical soundness (whether the proposed methodology logically supports the hypothesis) and feasibility (whether the proposal could be executed within 3 months by a small research team), both on 1–5 scales. Evaluation is single-blind: experts see anonymized proposals with system identifiers removed. We report inter-rater reliability using intraclass correlation (ICC).

Results. ResearchOS proposals achieve an average completeness score of 0.81 (9.7 of 12 items adequate), compared to 0.68 for B4 (Graph Retrieval Only), 0.59 for B2 (RAG), and 0.72 for B7 (Human Expert Seed). The gap is most pronounced for items related to risk mitigation (0.87 vs. 0.40 for B2), computational budget estimation (0.80 vs. 0.33 for B1), and control conditions (0.83 vs. 0.47 for B3). On technical soundness, ResearchOS scores 4.0 versus 3.4 for B4 and 3.7 for B7. On feasibility, ResearchOS and B7 are comparable (3.8 vs. 3.9), both outperforming other automated methods. Expert inter-rater reliability is high (ICC $= 0.82$), indicating consistent evaluation. These results confirm that the structured composition process—particularly gap analysis and self-critique—translates into more complete and technically sound proposals.

4.4 Experiment 3: Future-Grounded Novelty

A fundamental question for any research idea generation system is whether it produces genuinely anticipatory ideas rather than merely recombining known concepts. We evaluate this through a time-split validation protocol inspired by trend prediction in science [13].

Setup. We construct the skill graph using only papers published on or before June 2024 (87 papers, 198 skills, 614 edges). We then run all systems to generate designs for 10 research goals. Ground-truth validation uses 34 papers published between July 2024 and March 2025, which were not seen during skill extraction or composition.

Hit levels. We define three levels of match between a generated design and a subsequently published paper, evaluated by a combination of embedding similarity and expert judgment:

- **Direction Hit**: the generated design addresses the same high-level research direction as the published paper (goal embedding cosine similarity > 0.8).
- **Method Hit**: the generated design proposes a mechanism substantially similar to the one used in the published paper (mechanism description embedding similarity > 0.75).

- **Composition Hit**: at least 70% of the skills composed in the generated design overlap with the techniques employed in the published paper, as determined by expert annotation.

Direction Hits indicate alignment with research trends; Method Hits indicate anticipation of specific technical approaches; Composition Hits indicate that the system predicted a particular combination of techniques.

Metrics. For each system, we report: **Hit Rate@**K, the fraction of the top-K generated designs (ranked by the system's own confidence) that match at least one future paper at each hit level; **Precision@**K, the fraction of future papers matched by at least one of the top-K designs; and **Coverage**, the fraction of all 34 future papers matched at any hit level across all designs. We additionally compute a random chance baseline by sampling 1,000 random skill compositions and measuring their hit rates.

Results. At $K = 5$, ResearchOS achieves Direction Hit Rate of 0.72, Method Hit Rate of 0.38, and Composition Hit Rate of 0.14. The next-best automated system (B4, Graph Retrieval Only) achieves 0.58, 0.22, and 0.06 respectively. The random chance baseline yields Direction Hit Rate of 0.31, Method Hit Rate of 0.05, and Composition Hit Rate of 0.01, confirming that the results are far above chance. The Human Expert Seed baseline achieves comparable Direction Hit Rate (0.69) but lower Method and Composition Hit Rates (0.29 and 0.08), suggesting that while experts identify promising directions, ResearchOS more precisely anticipates the specific mechanisms that materialize.

Coverage analysis reveals that ResearchOS designs collectively match 21 of 34 future papers at the Direction level (62%), 11 at the Method level (32%), and 4 at the Composition level (12%). Stratified by sub-direction, performance is strongest for papers on reasoning verification and self-correction (Method Hit Rate 0.52) and weakest for papers introducing entirely new problem formulations (Method Hit Rate 0.11), which is expected since the system composes from existing skills rather than inventing new primitives.

4.5 Experiment 4: Small-Scale Execution

To ground our evaluation beyond automated scoring, we select three ResearchOS designs and two baseline designs (one from B2, one from B7) for small-scale execution. Independent researchers (graduate students not involved in system development) receive Level C Executable Experiment Plans and attempt to implement and run each design over a two-week period.

Metrics. We measure three dimensions of execution quality:

- **Executability**: the number of blockage points—steps where the researcher could not proceed without significant deviation from the plan. Lower is better.
- **Prediction Accuracy**: the relative error between predicted and actual performance on specified metrics, computed as $|\text{actual} - \text{predicted}|/|\text{predicted}|$. Lower indicates better calibration.
- **Discovery Value**: expert rating (1–5) of whether the executed experiment produced a finding that would merit inclusion in a workshop paper.

Case studies. We summarize the three ResearchOS executions:

Case 1: Adaptive Verification Depth. This design composes chain-of-thought generation [27] with a learned verification policy that dynamically adjusts the number of verification steps based on problem difficulty. The plan was fully executable with zero blockage points. Actual accuracy improvement on GSM8K ($+3.1\%$) was close to the predicted $+4.0\%$ (relative error 22.5%). The verification policy learned meaningful difficulty-aware behavior, rated 4/5 for discovery value.

Case 2: Conflict-Aware Ensemble Reasoning. This design combines self-consistency [26] with reflexion [22], using a conflict detection module to identify when these two strategies produce contradictory signals. One blockage point arose from underspecified conflict resolution criteria. Actual performance ($+2.8\%$ on MATH) fell short of the predicted $+5.2\%$ (relative error 46.2%), suggesting that the gap-bridging component overestimated synergy. Discovery value was rated 3/5.

Case 3: Hierarchical Decomposition with Skill Routing. This design layers problem decomposition with a routing mechanism that selects specialized reasoning skills per sub-problem. Two blockage

points arose from the routing mechanism's training data requirements, which were underestimated in the plan. Despite blockages, the design produced a functional system with promising preliminary results (discovery value 3/5).

The two baseline designs (B2: RAG-generated, B7: expert-seeded) encountered 3 and 1 blockage points respectively. The expert-seeded design had the best prediction accuracy (relative error 18.4%) but the lowest novelty among all executed designs. Overall, ResearchOS designs are executable with moderate blockage rates and produce findings of genuine, if preliminary, research interest.

4.6 Experiment 5: Judge Reliability

Automated evaluation of research quality raises well-documented concerns about LLM judge reliability [29]. We conduct four analyses to characterize and mitigate these concerns.

Human-LLM agreement. On the 30-sample calibration set, Cohen's κ between each LLM judge and the majority human label is: Claude Sonnet $\kappa = 0.71$, GPT-4 $\kappa = 0.68$, Gemini Pro $\kappa = 0.63$. All exceed the $\kappa \geq 0.60$ threshold for substantial agreement. Agreement is highest for feasibility ($\kappa = 0.78$ averaged across judges) and lowest for novelty ($\kappa = 0.58$), consistent with novelty being the most subjective dimension.

Position bias. In pairwise comparisons, we measure the flip rate—the fraction of judgments that change when the presentation order of two designs is swapped. The average flip rate is 11.3% across all judges, with Claude Sonnet exhibiting the lowest rate (8.7%) and Gemini Pro the highest (14.2%). Majority voting across three judges reduces the effective flip rate to 4.1%, and our tiebreaker protocol resolves all remaining inconsistencies.

Self-preference bias. To test whether LLM judges favor designs generated by the same model family, we compare scores assigned when the generator and judge share a model family (*matched*) versus when they differ (*unmatched*). The matched condition shows a score inflation of $+0.18$ on average across dimensions (statistically significant, $p < 0.05$ by paired t-test). This effect is mitigated by our cross-model majority voting protocol: when three judges vote, at most one can be matched with the generator, limiting the bias impact.

Inter-model consistency. Fleiss' κ [8] across the three LLM judges is 0.64 for pairwise novelty comparisons and 0.71 for significance comparisons. For Likert scoring, Fleiss' κ ranges from 0.59 (cost-awareness) to 0.76 (clarity). These values indicate moderate to substantial agreement, comparable to inter-rater reliability reported in peer review studies.

4.7 Ablation Studies

We conduct systematic ablations to isolate the contribution of each major component of ResearchOS. All ablations are evaluated on the full set of 20 research goals using the same judging protocol as Experiment 1.

Component ablations. We evaluate four ablation variants:

- $-$**Gap**: remove gap analysis and bridging; compose only from retrieved skills without synthesizing novel connective mechanisms.
- $-$**Critique**: remove the iterative self-critique loop; use the first-pass composition directly.
- $-$**Validator**: remove the constraint validator; allow compositions that may contain unresolved conflicts or unsatisfied prerequisites.
- $-$**Conflict**: remove CONFLICTS and SUBSTITUTES edges from the graph, retaining only positive relations.

Skill quality sensitivity. We additionally test robustness to skill extraction quality by injecting controlled noise into skill schemas: replacing 10%, 20%, and 30% of skill field values with plausible but incorrect alternatives (e.g., swapping assumptions between skills, randomizing empirical gain magnitudes).

Table 2: Ablation results. Values show mean Likert scores across 20 goals. Δ columns indicate change from the full ResearchOS system.

Variant	Novelty	Feasibility	Tech. Qual.	Completeness	Repr.	Risk-Aw.
Full ResearchOS	4.1	3.8	3.8	3.9	3.7	3.6
−Gap	3.4 (−0.7)	3.7 (−0.1)	3.5 (−0.3)	3.5 (−0.4)	3.4 (−0.3)	3.3 (−0.3)
−Critique	3.9 (−0.2)	3.6 (−0.2)	3.3 (−0.5)	3.5 (−0.4)	3.3 (−0.4)	3.1 (−0.5)
−Validator	4.2 (+0.1)	3.2 (−0.6)	3.0 (−0.8)	3.4 (−0.5)	3.2 (−0.5)	2.8 (−0.8)
−Conflict	4.0 (−0.1)	3.5 (−0.3)	3.4 (−0.4)	3.7 (−0.2)	3.5 (−0.2)	3.2 (−0.4)
10% noise	3.9 (−0.2)	3.7 (−0.1)	3.6 (−0.2)	3.7 (−0.2)	3.5 (−0.2)	3.4 (−0.2)
20% noise	3.6 (−0.5)	3.5 (−0.3)	3.4 (−0.4)	3.4 (−0.5)	3.2 (−0.5)	3.1 (−0.5)
30% noise	3.3 (−0.8)	3.2 (−0.6)	3.1 (−0.7)	3.1 (−0.8)	2.9 (−0.8)	2.8 (−0.8)

Results. Table 2 presents the ablation results. Removing gap analysis (−Gap) causes the largest drop in novelty (-0.7 Elo-equivalent Likert points), confirming that bridging between existing skills is the primary source of novel contributions. Removing self-critique (−Critique) reduces technical quality (-0.5) and completeness (-0.4) but has minimal effect on novelty, suggesting that critique primarily refines rather than generates ideas. Removing the validator (−Validator) produces designs with higher surface-level novelty ($+0.1$) but substantially lower feasibility (-0.6) and technical quality (-0.8), as invalid compositions are no longer filtered. Removing conflict edges (−Conflict) causes a moderate drop in feasibility (-0.3) and technical quality (-0.4), as the system occasionally combines incompatible techniques.

Noise injection degrades performance gracefully: at 10% noise, average scores drop by 0.1–0.2 across dimensions; at 20%, drops reach 0.3–0.5; at 30%, the system's performance approaches that of the Flat Skill List baseline (B3), indicating that graph-based composition provides diminishing returns when the underlying skill descriptions are substantially corrupted.

5 Discussion

Structure of the method space. The Research Skill Graph reveals several structural properties of the LLM reasoning method space. Skill degree distributions follow a power law: a small number of foundational skills (chain-of-thought prompting, self-consistency, in-context learning) participate in the majority of compositions, while most skills are specialized and sparsely connected. The graph exhibits modular community structure, with clusters corresponding to prompting strategies, verification methods, search algorithms, and training-time interventions. Interestingly, the most novel designs produced by ResearchOS tend to bridge between these clusters, combining skills from different communities—an observation consistent with theories of innovation through analogical transfer [10].

Failure cases. Composition fails in several characteristic ways. First, when composed skills rest on incompatible assumptions (e.g., one skill assumes access to ground-truth intermediate steps while another assumes a fully unsupervised setting), the resulting design contains internal contradictions that the validator catches in approximately 73% of cases but misses in the remainder. Second, the system occasionally produces *over-complex* designs that compose 6–8 skills into an intricate pipeline whose interaction effects are difficult to predict. These designs score well on novelty but poorly on feasibility. Third, gap-bridging sometimes introduces speculative connective mechanisms that lack empirical grounding, inflating novelty scores while reducing technical soundness—an effect visible in the −Validator ablation.

Limitations. Our evaluation is subject to several important limitations. *Domain specificity*: we evaluate exclusively on LLM reasoning, a domain with particularly modular and composable methods. Extension to domains with less modular structure (e.g., hardware design, wet-lab biology) may require fundamentally different skill representations. *Skill extraction quality*: the system's performance depends heavily on the accuracy of extracted skill schemas; our noise injection experiments show that performance degrades substantially beyond 20% corruption. The 20% human calibration rate, while practical, leaves room for systematic extraction errors. *LLM judge limitations*: despite our

multi-judge protocol, automated evaluation of research novelty remains an open problem [23]. Our human-LLM agreement analysis shows that novelty is the dimension with the lowest inter-rater agreement, suggesting that absolute novelty scores should be interpreted with caution. *Execution scale*: our small-scale execution study covers only five designs, which is insufficient to draw general conclusions about execution success rates.

Broader impact. ResearchOS could accelerate the research ideation process, potentially democratizing access to structured research planning for researchers with less experience navigating large literatures. However, we note two risks. First, widespread adoption of composition-based ideation could *homogenize* research directions, as all users draw from the same skill graph and similar composition heuristics. Second, the system may create a false sense of completeness: a well-formatted Research Proposal is not a substitute for the deep domain understanding needed to execute research successfully. Generated designs should be treated as structured hypotheses to be critically evaluated, not as validated conclusions.

Ethics. We emphasize that ResearchOS outputs are *candidate designs*, not finished research. Users bear responsibility for evaluating feasibility, ethical implications, and potential harms of any design before execution. The system does not assess whether a proposed method could cause harm through its application (e.g., improved reasoning applied to generating misinformation). We advocate for human-in-the-loop deployment where experts review and filter generated designs before they enter the research pipeline.

6 Conclusion

We have presented ResearchOS, a framework that transforms unstructured scientific papers into composable *research skills*—structured primitives with defined functional interfaces—and organizes them into a Research Skill Graph with typed relations capturing prerequisite, enhancement, substitution, conflict, composition, and refinement dependencies. A constraint-aware composition engine leverages this graph to generate novel method designs through goal decomposition, candidate retrieval, directed acyclic graph construction, gap-bridging, and iterative self-critique, producing three levels of progressively actionable output: Method Design Cards, Research Proposals, and Executable Experiment Plans.

Our experiments yield four key findings. First, structured composition over a skill graph significantly outperforms direct generation, retrieval-augmented generation, and unstructured skill lists, demonstrating that explicit relational structure is essential for high-quality method design. Second, graph relations—particularly conflict and substitution edges—are critical for producing feasible designs; removing them leads to compositions with internal contradictions. Third, gap-bridging is the primary driver of genuine novelty, synthesizing connective mechanisms that go beyond recombination of existing techniques. Fourth, future-grounded evaluation via time-split validation confirms that ResearchOS designs anticipate research directions found in subsequently published papers, with Method Hit Rates substantially above both baselines and random chance.

Looking ahead, we see three directions for extending this work: broadening skill extraction to additional research domains beyond LLM reasoning, closing the loop between design generation and automated execution [16], and developing community-maintained skill repositories that grow and improve as the research literature evolves.

References

[1] Anthropic. The Claude model family. 2024.

[2] Jinheon Baek, Sujin Jang, and Sung Ju Hwang. ResearchAgent: Iterative research idea generation over scientific literature with large language models. *arXiv preprint arXiv:2404.07738*, 2024.

[3] Maciej Besta, Nils Blach, Ales Kubicek, et al. Graph of thoughts: Solving elaborate problems with large language models. In *AAAI*, 2024.

[4] Boyuan Chen et al. AutoML in the age of large language models: Current challenges, future opportunities and risks. In *KDD*, 2024.

[5] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.

[6] Tianjun Du et al. Hindsight: Evaluate research ideas with retrospective analysis. *arXiv preprint*, 2024.

[7] Arpad E Elo. The rating of chessplayers, past and present. Arco, 1978.

[8] Joseph L Fleiss. Measuring nominal scale agreement among many raters. *Psychological Bulletin*, 76(5):378–382, 1971.

[9] Sebastian Gottifredi and Google Deepmind. AI co-scientist. *Google DeepMind Technical Report*, 2024.

[10] Tom Hope, Joel Chan, Aniket Kittur, and Dafna Shahaf. Accelerating innovation through analogy mining. In *KDD*, 2017.

[11] Qian Huang, Jian Hu, et al. MLR-Bench: A benchmark for machine learning research idea generation and evaluation. *arXiv preprint*, 2024.

[12] Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. *Automated Machine Learning: Methods, Systems, Challenges*. Springer, 2019.

[13] Mario Krenn and Anton Zeilinger. Predicting research trends with semantic and neural networks. *PNAS*, 2022.

[14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.

[15] Long Li et al. Chain of ideas: Revolutionizing research via novel idea development with LLM agents. *arXiv preprint arXiv:2410.13185*, 2024.

[16] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

[17] Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. Chameleon: Plug-and-play compositional reasoning with large language models. *Advances in Neural Information Processing Systems*, 36, 2023.

[18] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2024.

[19] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023.

[20] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2023.

[21] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. *Advances in Neural Information Processing Systems*, 36, 2023.

[22] Noah Shinn, Federico Cassano, Ashwin Gopinath, et al. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

[23] Chenglei Si, Diyi Yang, et al. Can LLMs generate novel research ideas? A large-scale human study with 100+ NLP researchers. *arXiv preprint arXiv:2409.04109*, 2024.

[24] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

[25] Qingyun Wang, Doug Downey, Heng Ji, and Tom Hope. SciMON: Scientific inspiration machines optimized for novelty. *ACL*, 2024.

[26] Xuezhi Wang, Jason Wei, Dale Schuurmans, et al. Self-consistency improves chain of thought reasoning in language models. In *ICLR*, 2023.

[27] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

[28] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2023.

[29] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *NeurIPS*, 2024.

A Research Skill Schema Specification

Table 3 presents the complete field specification for a research skill primitive. Every extracted skill must populate all required fields; optional fields are marked accordingly.

Table 3: Research Skill Schema. All fields are required unless marked optional.

Field	**Type & Constraints**	**Example (Chain-of-Thought)**
`id`	String, unique	`skill_cot_001`
`name`	String, $\leq$10 words	Chain-of-Thought Prompting
`goal`	String, 1–2 sentences	Improve LLM accuracy on complex reasoning tasks
`mechanism`	String, $\leq$3 sentences, must describe *operations* not outcomes	Insert intermediate reasoning steps in prompts to guide step-by-step derivation
`assumptions`	List[String]	Model $\geq$60B params; task is multi-step decomposable
`inputs`	List[String], typed	Prompt template, exemplar QA pairs, task query
`outputs`	List[String], typed	Model output with reasoning chain
`level`	Enum: {training, inference, data, architecture, prompting}	prompting
`resources`	String	No training; inference tokens $\times$3–5
`empirical_gains`	List[String], with conditions	GSM8K: 58%$\rightarrow$75% (PaLM 540B)
`failure_modes`	List[String]	Unfaithful chains; small models show no gain
`compatibility`	List[skill_id]	self-consistency, verification
`conflict`	List[skill_id]	direct-answer prompting
`source_papers`	List[String]	[27]
`tags`	List[String], hierarchical	[reasoning, prompting, inference-time]

Granularity Principle. A valid skill must satisfy three criteria: (1) it can be described independently of its source paper's specific experimental context; (2) it can be replaced by an alternative skill serving the same sub-goal; and (3) it has well-defined inputs and outputs that enable composition with other skills. Operationally, if one could write a self-contained paragraph in a new paper's method section describing the technique, it qualifies as a skill.

B Skill Extraction Prompt Template

The following is the core prompt used in the Paper-to-Skill Compiler (Section 3.3):

```
You are a research method analyst. Given the following paper
sections, extract all distinct research skills (method primitives)
that represent independently reusable techniques.

For each skill, fill in ALL fields of the following schema:

- name: (short, ≤10 words)
- goal: (what problem does this technique solve?)
- mechanism: (HOW does it work? Describe the operation, not the
outcome. Must be specific enough that someone could reimplement
it.)
- assumptions: (what must be true for this to work?)
- inputs: (what does it take as input? Be specific about types.)
- outputs: (what does it produce?)
- level: (training-time / inference-time / data / architecture /
prompting)
- required_resources: (compute, data, or other requirements)
- empirical_gains: (quantitative results WITH conditions)
- failure_modes: (when does this NOT work?)
- compatibility: (what other methods work well with this?)
- conflict: (what methods conflict with this?)

IMPORTANT: The mechanism field must describe OPERATIONS, not
outcomes. BAD: "We propose a novel method that improves reasoning."
GOOD: "Insert step-by-step reasoning demonstrations before the query,
each showing intermediate derivation steps."

Paper sections:
[INTRODUCTION]: {intro}
[METHOD]: {method}
[EXPERIMENTS]: {experiments}
[RELATED WORK]: {related_work}
[LIMITATIONS]: {limitations}
```

C Edge Inference Prompt Template

```
Given two research skills, determine their relationship(s). Choose
from:

- PREREQUISITE(A, B): Using B requires A to be in place first
- ENHANCES(A, B): A improves B's effectiveness
- SUBSTITUTES(A, B): A and B solve the same sub-problem,
interchangeable
- CONFLICTS(A, B): A and B cannot be used together
- COMPOSES(A, B): A and B combine to form a higher-level method
- REFINES(A, B): B is an improved version of A
- NONE: No significant relationship

For each identified relationship, provide:
- type: the relationship type
- confidence: 0.0-1.0
- evidence: brief justification

Skill A: {skill_a_schema}
Skill B: {skill_b_schema}
```

D Evaluation Rubrics

We provide structured rubrics for each evaluation dimension. Each rubric is provided to the LLM judge along with the design being evaluated.

D.1 Novelty Rubric

- **1 – Duplicate:** The proposed method is essentially identical to an existing published method. No new combination or mechanism.
- **2 – Minor Variation:** The method makes superficial changes to an existing approach (e.g., different hyperparameters, trivial reordering of known steps).
- **3 – Non-trivial Combination:** The method combines existing techniques in a way that has not been explicitly explored, but the combination is relatively straightforward.
- **4 – Novel Composition:** The method includes a genuinely new bridging mechanism or an unexpected cross-domain combination. The composition produces emergent properties beyond the sum of its parts.
- **5 – Highly Novel:** The method introduces a fundamentally new mechanism or a paradigm-shifting combination that opens new research directions.

D.2 Feasibility Rubric

- **1 – Infeasible:** Critical prerequisites are unmet, or required resources are clearly unavailable. Contains fundamental technical contradictions.
- **2 – Major Barriers:** Significant implementation challenges exist with no clear solution path. May require breakthroughs in other areas first.
- **3 – Feasible with Effort:** Implementable but requires substantial engineering effort or moderate resources. Some open questions remain.
- **4 – Readily Feasible:** Clear implementation path with existing tools and moderate resources. Minor uncertainties only.
- **5 – Immediately Executable:** Can be implemented with existing code, data, and compute in days. All components are well-understood.

D.3 Significance Rubric

- **1 – Negligible:** Expected improvement is marginal ($<1\%$) or addresses an unimportant problem.
- **2 – Incremental:** Expected improvement is modest (1–5%) on established benchmarks for a known problem.
- **3 – Moderate:** Expected improvement is substantial (5–15%) or addresses a recognized gap in the field.
- **4 – Significant:** Expected to meaningfully advance the state-of-the-art or establish a new capability.
- **5 – Breakthrough:** If successful, would fundamentally change how the community approaches the problem.

D.4 Technical Quality Rubric

- **1:** Contains logical contradictions or fundamentally flawed reasoning.
- **2:** Reasoning has gaps; some assumptions are unjustified or inconsistent.
- **3:** Reasoning is mostly sound but has minor issues; assumptions are reasonable.
- **4:** Technically rigorous with well-justified assumptions and clear logic.
- **5:** Exceptionally rigorous; considers edge cases, provides theoretical grounding.

E Pairwise Comparison Prompt

```
You are evaluating two research method designs on {METRIC}. Read
both designs carefully, then determine which is better on this
specific dimension.
Rubric for {METRIC}:
{RUBRIC}
Design A:
{DESIGN_A}
Design B:
{DESIGN_B}
First, analyze each design on the {METRIC} dimension in 2-3
sentences. Then state your verdict: "A is better", "B is better",
or "Tie". Provide a brief justification for your choice.
Output format:
Analysis of A: ...
Analysis of B: ...
Verdict: [A is better / B is better / Tie]
Justification: ...
```

F Research Goals Used in Evaluation

Table 4 lists the 20 research goals used in our experiments.

Table 4: Research goals for evaluation. Goals 1–10 are manually designed; Goals 11–20 are extracted from paper future work sections.

ID	Type	Research Goal
G1	Manual	Improve faithfulness of LLM reasoning chains in multi-step mathematical problems
G2	Manual	Reduce inference latency of chain-of-thought reasoning without sacrificing accuracy
G3	Manual	Enable reliable reasoning in LLMs with fewer than 10B parameters
G4	Manual	Improve LLM performance on multi-hop factual reasoning over long contexts
G5	Manual	Design self-correcting reasoning that detects and fixes its own errors
G6	Manual	Combine symbolic and neural reasoning for improved compositionality
G7	Manual	Improve reasoning robustness to adversarial or misleading premises
G8	Manual	Enable efficient test-time compute scaling for reasoning tasks
G9	Manual	Transfer reasoning capabilities across domains without domain-specific training
G10	Manual	Improve calibration of LLM confidence in reasoning outputs
G11	Extracted	Extend step-level verification to natural language reasoning beyond math
G12	Extracted	Develop reasoning methods that handle ambiguous or under-specified problems
G13	Extracted	Combine retrieval-augmented generation with structured reasoning
G14	Extracted	Reduce hallucination in reasoning chains through grounding mechanisms
G15	Extracted	Develop adaptive reasoning depth based on problem complexity
G16	Extracted	Improve multi-agent debate for complex reasoning with disagreement resolution
G17	Extracted	Design curriculum learning strategies for reasoning skill acquisition
G18	Extracted	Combine process reward models with outcome reward models for reasoning
G19	Extracted	Enable reasoning with noisy or incomplete evidence
G20	Extracted	Develop methods for compositional generalization in reasoning

G Proposal Completeness Checklist

The 12-item checklist used to evaluate Level B proposals:

1. **Hypothesis:** A clear, falsifiable hypothesis is stated.
2. **Independent Variables:** All manipulated variables are specified.
3. **Dependent Variables:** All measured outcomes are defined.
4. **Controls:** Controlled variables and control conditions are listed.
5. **Metrics:** Quantitative evaluation metrics are specified.
6. **Datasets:** Specific datasets/benchmarks are named with split information.
7. **Baselines:** Comparison methods are identified.
8. **Ablations:** Ablation conditions are designed to isolate contributions.
9. **Implementation Notes:** Key implementation details are provided.
10. **Resource Estimate:** Compute, data, and time requirements are estimated.
11. **Risk Analysis:** Potential failure modes and mitigation strategies are discussed.
12. **Expected Outcomes:** Predicted results and interpretation of negative results are provided.

H Additional Design Examples

We present two additional generated designs to illustrate the range of compositions produced by RESEARCHOS.

Example 1: Adaptive-Depth Verified Reasoning.

- **Goal:** G15 — Develop adaptive reasoning depth based on problem complexity.
- **Selected Skills:** Complexity classifier, early-exit mechanism, step-level verifier, confidence calibrator.
- **Novel Bridging:** A *complexity-conditional routing* mechanism that uses a lightweight classifier to predict problem difficulty, then allocates reasoning depth (number of CoT steps) proportionally. The step-level verifier provides feedback to the classifier, creating a calibration loop.
- **Expected Gain:** 40–60% latency reduction on easy problems with $<2\%$ accuracy drop; maintained accuracy on hard problems.
- **Key Risk:** Misclassification of hard problems as easy leads to truncated reasoning and accuracy loss.

Example 2: Retrieval-Grounded Debate.

- **Goal:** G16 — Improve multi-agent debate with disagreement resolution.
- **Selected Skills:** Multi-agent debate, evidence retrieval, argument scoring, consensus mechanism.
- **Novel Bridging:** When debate agents disagree, trigger targeted retrieval for the disputed claim. Each agent must ground its argument in retrieved evidence, and a *evidence-weighted voting* mechanism resolves disputes by weighting votes by the relevance and reliability of cited evidence.
- **Expected Gain:** Reduced debate rounds ($3 \rightarrow 2$ average); improved factual accuracy of conclusions by 8–12%.
- **Key Risk:** Retrieval corpus may not contain evidence for novel reasoning steps, causing deadlock.